

**LAPORAN PRAKTIKUM PEMROGRAMAN
BERORIENTASI OBJEK**

PEKAN 2

**MEMBUAT FUNGSI CRUD (CREATE, READ, UPDATE,
DELETE) USER DENGAN DATABASE MYSQL**



Oleh :

Jiyad Rifqi Pasaribu

NIM 2411533003

**MATA KULIAH PRAKTEKUM PEMROGRAMAN BERORIENTASI
OBJEK**

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

**FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG**

A. Pendahuluan

Dalam pengembangan aplikasi perangkat lunak saat ini, pengelolaan data sangat penting. Hampir semua aplikasi, dari yang sederhana hingga yang besar, perlu bisa berhubungan dengan database untuk menyimpan, mengambil, mengubah, dan menghapus data. Ini dikenal dengan istilah CRUD, yang berarti Create (Membuat), Read (Membaca), Update (Memperbarui), dan Delete (Menghapus). Menguasai cara membuat fungsi CRUD adalah langkah awal yang penting bagi pengembang agar aplikasi yang dibuat menjadi dinamis dan berguna.

Laporan ini membahas pembuatan aplikasi desktop untuk mengelola data pengguna dengan menerapkan keempat fungsi CRUD tersebut. Contoh ini dipilih karena pengelolaan data pengguna adalah fitur yang sering ada di banyak jenis aplikasi, seperti aplikasi kasir, sistem keanggotaan, atau media sosial.

B. Tujuan

Tujuan dari praktikum ini adalah sebagai berikut:

1. Menjelaskan struktur dan bagian-bagian yang dipakai untuk membuat aplikasi, seperti paket untuk koneksi (config), model tabel (table), penyimpanan data (DAO), dan tampilan pengguna (ui).
2. Memberikan langkah-langkah mudah untuk membuat fungsi CRUD menggunakan PreparedStatement agar interaksi dengan database lebih aman dan cepat.
3. Menghasilkan sebuah aplikasi fungsional yang dapat melakukan manajemen data pengguna secara visual.

C. Langkah – langkah Pengerjaan

1. Download my sql untuk bisa mengakses database yang akan di pakai nantinya, setelah itu download mysql connector agar java bisa terhubung ke database.
2. Setelah selesai di download, tambahkan MySQL Connector kedalam project dengan cara, klik kanan pada JRE System Library → Build Path → Configure Build Path...
3. Lalu pilih Libraries → Classpath.
4. Sebelum kita klik Add External JARs, ekstrak terlebih dahulu file MySQL Connector yang telah di download tadi agar bertipe Jar.
5. Jika sudah, klik Add External JARs, lalu pilih file yang sudah di ekstrak tadi, lalu klik Apply and Close.
6. Jika berhasil maka akan generate folder Referenced Libraries pada project yang berisi MySQL Connector.

>  mysql-connector-j-9.0.0.jar - D:\mysql-connector-j-9.0.0\m

7. Buka phpmyadmin dengan mengakses link berikut “http://localhost/phpmyadmin”.
8. Setelah di buka, silahkan buatdatabase dengan nama laundry_apps.

Basis data

 **Buat basis data** 

Buat

9. Buat tabel user pada database tadi dan sesuaikan jumlah kolom nya dengan yang ada pada gambar.

Nama tabel

Jumlah kolom

Buat

10. Setelah jadi, maka akan muncul tampilan seperti berikut. Isi sesuai dengan gambar di bawah.

Nama	Jenis	Panjang/Nilai	Bawaan	Penyortiran	Atribut	Tak Terlihat	Indeks
id	INT		Tidak ada			☐	PRIMARY
name	VARCHAR	128	Tidak ada			☐	
username	VARCHAR	128	Tidak ada			☐	
password	TEXT		Tidak ada			☐	

Komentar tabel: Collation: Storage Engine: InnoDB

PARTITION definition: (Expression or column list)

Partitions:

Pratinjau SQL

11. Untuk bisa menyambungkan kodingan ke database buat package baru pada src dengan nama config yang berguna untuk mengatur database yang akan di pakai.

12. Setelah itu buat class baru pada package config dengan nama Database.

13. Setelah class berhasil di buat, isi class tadi dengan kodingan di bawah ini yang mana nanti akan menghubungkan ke database yang di buat tadi.

```
package config;

import java.sql.*;
import javax.swing.JOptionPane;

import java.sql.Connection;

public class Database {
    Connection conn;

    public static Connection koneksi() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = (Connection) DriverManager.getConnection("jdbc:mysql://localhost/laundry_apps", "root", "");
            return conn;
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null, e);
            return null;
        }
    }
}
```

14. Setelah tadi sudah menghubungkan ke database, sekarang akan membuat tampilan CRUD user.

15. Buat class baru untuk tampilan user dengan nama UserFrame pada package ui.

The screenshot shows a Java Swing IDE with a 'Components' panel on the left, a 'Palette' panel in the middle, and a 'Properties' panel on the right. The main canvas displays a window titled 'USERS' with a light blue header. Inside the window, there are three text input fields labeled 'Nama', 'Username', and 'Password'. Below these fields are four buttons: 'Save' (green), 'Update' (blue), 'Delete' (red), and 'Cancel' (yellow). The 'Properties' panel shows the 'Name' property of the selected component.

16. Untuk membuat table model perlu membuat package baru dengan nama table.
17. Buat class baru pada package yang di buat tadi dengan nama TableUser, lalu isikan kodingan berikut pada class tersebut.

```
package table;

import javax.swing.table.AbstractTableModel;

import model.User;

import java.util.List;

public class TableUser extends AbstractTableModel {
    List<User> ls;
    private String[] columnNames = {"ID", "Name", "Username", "Password"};

    public TableUser(List<User> ls) {
        this.ls = ls;
    }

    @Override
    public int getRowCount() {
        return ls.size();
    }

    @Override
    public int getColumnCount() {
        return 4;
    }

    @Override
    public String getColumnName(int column) {
        return columnNames[column];
    }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        switch (columnIndex) {
            case 0:
                return ls.get(rowIndex).getId();
            case 1:
                return ls.get(rowIndex).getName();
            case 2:
                return ls.get(rowIndex).getUsername();
            case 3:
                return ls.get(rowIndex).getPassword();
            default:
                return null;
        }
    }
}
```

18. Selanjutnya membuat fungsi DAO dengan membuat package baru bernama DAO.
19. Buat interface pada package tersebut dengan nama UserDao, lalu isi dengan kodingan berikut.

```
package DAO;

import java.util.List;

import model.User;

public interface UserDao {
    void save(User user);
    public List<User> show();
    public void delete(String id);
    public void update(User user);
}
```

20. Untuk bisa menggunakan fungsi DAO buat class bernama UserRepo di package DAO untuk bisa mengimplementasikan interface tadi.

```
package DAO;

import java.util.logging.Level;
import java.sql.*;
import java.util.*;
import java.util.logging.Logger;

import DAO.UserDao;
import config.Database;
import model.User;

public class UserRepo implements UserDao {

    private Connection connection;
    final String insert = "INSERT INTO user (name, username, password) VALUES (?, ?, ?);";
    final String select = "SELECT * FROM user;";
    final String delete = "DELETE FROM user WHERE id=?;";
    final String update = "UPDATE user SET name=?, username=?, password=? WHERE id=?;";

    public UserRepo() {
        connection = (Connection) Database.koneksi();
    }

    @Override
    public void save(User user) {
        PreparedStatement st = null;
        try {
            st = connection.prepareStatement(insert);
            st.setString(1, user.getName());
            st.setString(2, user.getUsername());
            st.setString(3, user.getPassword());
            st.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try {
                st.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    @Override
    public List<User> show() {
        List<User> ls = null;
        try {
            ls = new ArrayList<User>();
            Statement st = connection.createStatement();
            ResultSet rs = st.executeQuery(select);
            while(rs.next()) {
                User user = new User();
                user.setId(rs.getString("id"));
                user.setName(rs.getString("name"));
                user.setUsername(rs.getString("username"));
                user.setPassword(rs.getString("password"));
                ls.add(user);
            }
        } catch (SQLException e) {
            Logger.getLogger(UserDao.class.getName()).log(Level.SEVERE, null, e);
        }
        return ls;
    }

    @Override
    public void update(User user) {
        PreparedStatement st = null;
        try {
            st = connection.prepareStatement(update);
            st.setString(1, user.getName());
            st.setString(2, user.getUsername());
            st.setString(3, user.getPassword());
            st.setString(4, user.getId());
            st.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try {
                st.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```

@Override
public void delete(String id) {
    PreparedStatement st = null;
    try {
        st = connection.prepareStatement(delete);
        st.setString(1, id);
        st.executeUpdate();

    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        try {
            st.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
}

```

D. KESIMPULAN

Dari hasil pengerjaan dapat disimpulkan bahwa program pengelolaan data pengguna ini sudah berhasil dibuat menggunakan Java dan database MySQL. Aplikasi ini mampu menjalankan fungsi utama seperti menambah, melihat, mengubah, dan menghapus data pengguna langsung dari tampilan. Kode program dipisah rapi antara bagian tampilan dan bagian yang berhubungan dengan database, sehingga sistem menjadi lebih teratur, mudah dikembangkan, dan lebih aman.